



A Thesis on-

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

This thesis is submitted to the department of Computer Science & Engineering in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science & Engineering

Under the Course of-

Course Title: Thesis/Project Work
Course Code: CSE 4000(B)

Thesis Supervisor-

Name: Mst. Sahela Rahman
Designation: Lecturer

Prepared by-

Name: Md. Riaz
ID/Registration No: 0322310105101024
Batch: 16th
Semester: 8th
Session: Spring - 2023

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY**

Date of submission: _____



A Thesis on-

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

This thesis is submitted to the department of Computer Science & Engineering in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science & Engineering

Signature of Supervisor

Signature of Student

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY**

Date of submission: _____

CERTIFICATION OF ORIGINALITY

I hereby declare that this thesis titled "AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics" is my own original work, carried out under the supervision of Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology. To the best of my knowledge, this thesis contains no material previously published or written by another person, except where due reference is made in the text. This work has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

Signature of Student

Md. Riaz

ID: 0322310105101024

CERTIFICATION OF APPROVAL

This is to certify that the research paper titled "AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

That it has been read, evaluated and accepted for fulfilment of the Academic Degree of Bachelor in Computer Science and Engineering (B.S.C. in CSE) at Pundra University of Science & Technology.

The acceptance decision is made based upon an independent review process that provides critically constructive feedback within a short turnaround time. This paper represents the author's independent work, critical analysis, and capability in undertaking literature research as per the academic policies and ethical standards of the university.

I certify that the candidate has completed the required research activities for the program and recommend that this piece of work as of acceptable standard for submission and for inclusion in the academic record of the University.

Mst. Sahela Rahman

Lecturer

Department of Computer Science and Engineering
Pundra University of Science & Technology

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology, for her continuous guidance, patience, and constructive feedback throughout the design, implementation, and evaluation of this research. Her insistence on precise, defensible claims shaped this thesis at every stage, from the formal safety proof to the honesty of its discussion of limitations.

I am also grateful to the faculty members of the Department of Computer Science & Engineering for the coursework and feedback that prepared me to undertake this research, and to my family for their patience and encouragement during the long implementation and evaluation cycles this thesis required.

Finally, I acknowledge the authors whose published research on natural language interfaces, text-to-SQL translation, and dashboard generation provided the foundation against which AEGIS's contributions are measured.

ABSTRACT

Relational databases hold data organizations need for decisions, but non-technical users often depend on developers to translate business questions into SQL. Natural-language interfaces try to close this gap, but many current systems still allow a language model to author executable SQL directly, making safety dependent on model behavior rather than system structure. This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), an architecture that restricts the model to intent extraction while deterministic software handles semantic mapping, permission enforcement, SQL compilation, validation, visualization selection, and widget persistence. AEGIS uses a closed semantic layer of approved metrics, dimensions, analytical patterns, and join paths, so the model never outputs SQL text. Instead, it produces a typed intent object that can be checked before any query is compiled (a typed intent object is a structured, checkable record of what the user asked for). The final evaluation uses two static nopCommerce corpora (nopCommerce is an open-source online-store platform that ships with its own admin reports): a 500-question natural-language benchmark, and nopCommerce's own twenty standard admin reports checked against the platform's own report implementations. On the 500-question live benchmark, AEGIS parsed 499 of 500 prompts, answered 423 and executed 422 of 425 supported requests, and rejected or clarified 72 of 75 realistic boundary requests. Against the twenty admin reports, every request compiled to SQL and fifteen reproduced the platform's own result set exactly, the remaining five agreeing on every value and differing only in row count and label column. The same model without the semantic layer, writing SQL directly, executed 365 of 425 supported questions but also answered 25 of the 75 questions it should have declined. Per-stage timing shows that substantially all latency is the model reading the question, while every deterministic stage after it completes in a few milliseconds. These results support the thesis's central claim that AEGIS is a bounded architecture for safe natural-language analytics over an approved semantic layer, not an unlimited text-to-SQL engine.

Table of Contents

Certification of Originality.....	i
Certification of Approval.....	ii
Acknowledgement.....	iii
Abstract.....	iv
Chapter 1: Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Research Novelty and Motivation.....	2
1.4 Objectives and Contributions.....	2
Chapter 2: Literature Review and Research Gap.....	4
2.1 Natural Language Interfaces to Databases.....	4
2.2 Neural and LLM-Based Text-to-SQL.....	4
2.3 Natural Language for Visualization and Dashboards.....	5
2.4 Applied Conversational Business Intelligence.....	6
2.5 Comparative Summary.....	7
2.6 Research Gap Analysis.....	7
Chapter 3: Methodology.....	8
3.1 Research Paradigm.....	8
3.2 Formative Study of Reporting Patterns.....	8
3.3 Design Principles.....	9
3.4 Formal Model.....	10
3.5 Threat Model.....	10
3.6 System Architecture.....	12
3.7 Semantic Layer Design.....	13
3.8 Intent Parsing with Dynamic Vocabulary Injection.....	15
3.9 Safe Query Compiler.....	17

3.10 Visualization Selector.....	20
3.11 Widget Persistence and Reuse.....	20
Chapter 4: Experimental Work.....	21
4.1 Implementation.....	21
4.2 Experimental Environment.....	22
4.3 Benchmark Dataset Construction.....	22
4.4 Baseline and Oracle Comparisons.....	23
4.5 Evaluation Procedure.....	25
Chapter 5: Results and Discussion.....	26
5.1 Evaluation Overview.....	26
5.2 500-Question Natural-Language Benchmark.....	26
5.3 Fidelity Against nopCommerce's Own Report Logic.....	28
5.4 Latency.....	29
5.5 Safety Evaluation.....	29
5.6 Comparison With Direct LLM-to-SQL.....	30
5.7 Interpretation.....	31
Chapter 6: Limitations and Future Work.....	32
6.1 Limitations.....	32
6.2 Future Work.....	33
Chapter 7: Conclusion.....	35
References.....	36

List of Figures

Figure 1: Design Science Research workflow.....	8
Figure 2: AEGIS architecture pipeline.....	13
Figure 3: Semantic layer modularity.....	14
Figure 4: Vocabulary injection and structured intent validation workflow.....	17
Figure 5: Two-layer SQL safety defence.....	19
Figure 6: Widget lifecycle and refresh model.....	20

List of Tables

Table 2.1: Comparative summary of related systems.....	7
Table 3.1: AEGIS reporting pattern taxonomy.....	9
Table 3.2: Semantic layer implementation contract.....	15
Table 3.3: AEGIS analytical patterns.....	18
Table 3.4: Visualization selector mapping.....	20
Table 4.1: Prototype module-to-architecture mapping.....	22
Table 4.2: Experimental setup.....	22
Table 4.3: Static evaluation datasets.....	23
Table 5.1: Evaluation artifacts.....	26
Table 5.2: 500-question live benchmark results.....	27
Table 5.3: nopCommerce standard admin report fidelity.....	28
Table 5.4: Per-stage latency over 425 supported questions.....	29
Table 5.5: Safety interpretation.....	30
Table 5.6: Measured comparison over the same 500 questions.....	30
Table 5.7: Structural comparison.....	31

CHAPTER 1

INTRODUCTION

1.1 Background

Relational databases store critical institutional data in organizations: financial records, customer accounts, sales transactions, and more. But accessing this data is uneven. Technical staff can write SQL queries to get any answer they need, while non-technical users have to wait for someone else to build them a report. This waiting is expensive. Analysis of enterprise reporting workflows shows that business users frequently wait days for new reports, and a recurring theme in institutional reporting is that many questions are variations of things already asked before: the same report with a different date range, or the same chart for a different department. These are not one-off questions; they are recurring reporting needs that should be served by saved, refreshable widgets rather than regenerated from scratch every time.

This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety). The system lets users describe reporting needs in plain English and produces dashboard widgets that can be saved, refreshed, and reused in daily work, without anyone writing SQL.

Natural language interfaces to databases (NLIDBs) try to solve this problem. A user should be able to ask "which categories have the highest refund rates this month?" and get a correct, visual answer without writing SQL. Benchmarks such as Spider [1] and BIRD [2] show clear progress. But there is still a gap between benchmark SQL generation and production-ready reporting, where answers must respect business definitions, permissions, safety rules, and reusable dashboard presentation.

1.2 Problem Statement

Three problems make up the gap between benchmark text-to-SQL accuracy and safe, production-ready natural-language reporting.

- **Safety:** Models may generate SQL directly, which can expose private data or create wrong joins.

- **Vocabulary mismatch:** Users ask with business terms such as "refund rate", while systems often expect column names.
- **No reusable widgets:** Most systems answer once and discard the result instead of saving refreshable dashboard widgets.

These problems are not mainly about building a smarter model. They are about designing the system around the model. AEGIS splits the work into stages. The LLM (large language model) understands the request and returns a structured description. Fixed rules and pre-approved templates then match business terms, build SQL, select the chart, and save the widget.

1.3 Research Novelty and Motivation

Existing text-to-SQL research asks how accurately a model can generate SQL from natural language. This thesis asks how a large language model can understand language while the system prevents it from generating executable SQL. The two pipelines differ structurally.

Classical NL-to-SQL: natural-language request, then model-authored SQL, then query result.

AEGIS: natural-language request, then intent extraction, semantic constraint, deterministic compilation, and a safe analytical artifact.

The contribution of this thesis is not better SQL generation. It is constrained analytical artifact generation. The design removes SQL generation from the LLM's role and gives the system safety and semantic-fidelity properties that prompt-only systems cannot guarantee.

AEGIS does not propose a new LLM. The model is interchangeable across APIs that expose an OpenAI-compatible /v1/chat/completions interface. Because the LLM's only contract with the rest of the system is to produce a typed JSON intent object, model upgrades can improve language understanding without changing the compiler or safety infrastructure. This model independence is part of the architecture. The evaluation in Chapter 5 used an LLM API exposed through an OpenAI-compatible interface; the SQL compiler and safety layer did not depend on that provider.

1.4 Objectives and Contributions

The objective of this thesis is to design and evaluate AEGIS, a constraint-based architecture that lets an LLM understand reporting requests without allowing it to generate executable SQL. Query construction, validation, visualization, and widget persistence are handled by deterministic system components.

The specific objectives and contributions are:

1. A natural-language analytics architecture where the LLM is structurally prevented from writing executable SQL.
2. A closed semantic layer that maps business questions to approved metrics, dimensions, filters, analytical patterns, and join paths.
3. A deterministic query compiler that builds SQL from approved templates instead of model-written SQL.
4. A two-layer SQL safety mechanism combining structural prevention with post-compilation validation.
5. A widget-oriented workflow that turns natural-language answers into reusable dashboard artifacts.
6. A static nopCommerce evaluation corpus consisting of a 500-question natural-language benchmark and nopCommerce's own twenty standard admin reports, checked against the platform's own report implementations rather than against an expected-answer set written by the authors of the system under test.
7. An empirical evaluation showing broad supported-request coverage, strong boundary rejection, a measured direct LLM-to-SQL baseline under identical conditions, per-stage latency, and a visible remaining Admin fidelity gap rather than a suspicious claim of perfect performance.

CHAPTER 2

LITERATURE REVIEW AND RESEARCH GAP

The literature review focuses on the work most directly comparable to AEGIS. It is organized as short review blocks so each source can be checked quickly by contribution, limitation, and the specific gap it leaves for this thesis.

2.1 Natural Language Interfaces to Databases

NLIDB surveys [3, 4]

- **Contribution:** Compare major natural-language database interface designs, including keyword, pattern, parsing, and grammar-based systems.
- **Limitations:** Safety is treated mostly as a secondary concern; even SQL-injection (running attacker-supplied text as SQL) discussion ends with generic filtering advice.
- **Gap for AEGIS:** No reviewed system makes SQL safety a structural design property.

NaLIR [5]

- **Contribution:** Uses an intermediate query tree and asks the user to choose between possible interpretations.
- **Limitations:** Correctness depends on user disambiguation, and the final output remains executable SQL.
- **Gap for AEGIS:** Ambiguity is handled after parsing, but unsafe or unauthorized queries are not prevented by design.

Veezoo [6]

- **Contribution:** Uses an editable Knowledge Graph that maps business language to database concepts.
- **Limitations:** The semantic layer improves usability, but the paper does not evaluate SQL safety or permission enforcement.
- **Gap for AEGIS:** A semantic layer exists in prior work, but not as a safety boundary.

2.2 Neural and LLM-Based Text-to-SQL

Spider and BIRD [1, 2]

- **Contribution:** Provide large text-to-SQL benchmarks for complex and cross-domain database questions.
- **Limitations:** They measure SQL correctness and execution accuracy, not whether a query should be allowed.
- **Gap for AEGIS:** Benchmark success does not equal database safety or business-policy compliance.

RAT-SQL [7]

- **Contribution:** Models schema relations directly so table and column linking can improve on complex schemas.
- **Limitations:** Errors still frequently come from wrong table or column selection.
- **Gap for AEGIS:** Better schema encoding reduces some mistakes but does not create an authorization boundary.

PICARD [8]

- **Contribution:** Constrains decoding so generated SQL is more likely to be syntactically valid.
- **Limitations:** The model still authors the SQL, and grammar validity does not prove the query is semantically safe.
- **Gap for AEGIS:** A valid SQL string can still express the wrong intent or access the wrong data.

G-SQL and TriSQL [9, 10]

- **Contribution:** Show two strong directions: rule-guided schema-aware translation and multi-stage LLM refinement.
- **Limitations:** G-SQL has limited qualitative evaluation, while TriSQL still depends on LLM-authored executable SQL.
- **Gap for AEGIS:** The closest prior work improves accuracy, but does not remove SQL generation from the LLM output space.

2.3 Natural Language for Visualization and Dashboards

nl4dv and DataTone [11, 12]

- **Contribution:** Map natural-language questions into visualization tasks and expose ambiguity instead of silently guessing.
- **Limitations:** They focus on visualization specification, not protected database execution or persistent dashboard widgets.
- **Gap for AEGIS:** Visualization systems solve chart selection but leave SQL safety and reuse outside the architecture.

DashBot [13]

- **Contribution:** Uses deep reinforcement learning and dashboard-design rules to compose multi-chart dashboards.
- **Limitations:** It has no natural-language-to-SQL stage and does not handle permission-controlled production database access.
- **Gap for AEGIS:** Dashboard composition is useful, but it does not solve safe language-to-data translation.

2.4 Applied Conversational Business Intelligence

Conversational BI assistants [14]

- **Contribution:** Demonstrate that LLM-based chat can be connected to dashboards and SQL tools.
- **Limitations:** Direct SQL execution through an LLM tool loop creates a large attack surface and is rarely evaluated adversarially.
- **Gap for AEGIS:** A production assistant needs a constrained execution layer, not only a conversational interface.

Explanatory dashboard analytics [15]

- **Contribution:** Shows that deterministic computation followed by LLM narration can improve reliability in business explanation tasks.
- **Limitations:** The work targets dashboard explanation, not safe SQL compilation or reusable widget creation.
- **Gap for AEGIS:** It supports AEGIS's choice to let deterministic code compute results while the LLM handles language.

2.5 Comparative Summary

Table 2.1: Comparative summary of the most closely related systems.

System	NL	Semantic	Safe SQL	Visual	Widget	Coverage	Evaluation
Spider/BIRD	Yes	-	-	-	-	-	Benchmark
RAT-SQL/ PICARD	Yes	-	Partial	-	-	-	Benchmark
G-SQL/TriSQL	Yes	Partial	Partial	-	-	-	Benchmark
NaLIR	Yes	-	-	-	-	-	User study
Veezoo	Yes	Yes	-	-	-	-	User study
nl4dv/DataTone	Yes	-	-	Yes	-	-	User study
DashBot	-	-	-	Yes	Partial	-	Synthetic
Conversational BI	Yes	-	-	Yes	-	-	Demo
AEGIS	Yes	Yes	Yes	Yes	Yes	Yes	Prototype

2.6 Research Gap Analysis

- **Safety gap:** Most systems measure answer accuracy, but do not evaluate unsafe SQL behavior.
- **Semantic-layer gap:** Semantic layers appear in prior work, but mainly for usability and matching, not as execution boundaries.
- **Visualization gap:** Visualization systems produce charts, but usually operate outside permission-controlled SQL execution.
- **Persistence gap:** Prior tools often answer one question at a time, while recurring business reports need refreshable outputs.
- **Thesis position:** AEGIS addresses these gaps through a fixed list of approved terms (a bounded vocabulary), a deterministic query compiler, safe visualization selector, and reusable widget model.

CHAPTER 3

METHODOLOGY

3.1 Research Paradigm

This thesis follows Design Science Research because the main contribution is a built and evaluated artifact, the AEGIS system. The method is summarized through three requirements:

- **Problem relevance:** Representative reporting requests motivate the need for the artifact.
- **Design structure:** The semantic layer, threat model, and compiler boundaries define how the artifact is designed.
- **Evaluation:** Static nopCommerce datasets evaluate supported-request coverage, boundary rejection, Admin fidelity, execution validity, and remaining implementation limits.

The artifact was refined through three build-evaluate cycles:

- **Cycle 1:** Built the initial semantic layer and compiler, then tested the core safety path.
- **Cycle 2:** Expanded coverage to the eleven analytical patterns and replaced manual synonym handling with vocabulary injection.
- **Cycle 3:** Added widget persistence and expanded the benchmark execution harness.

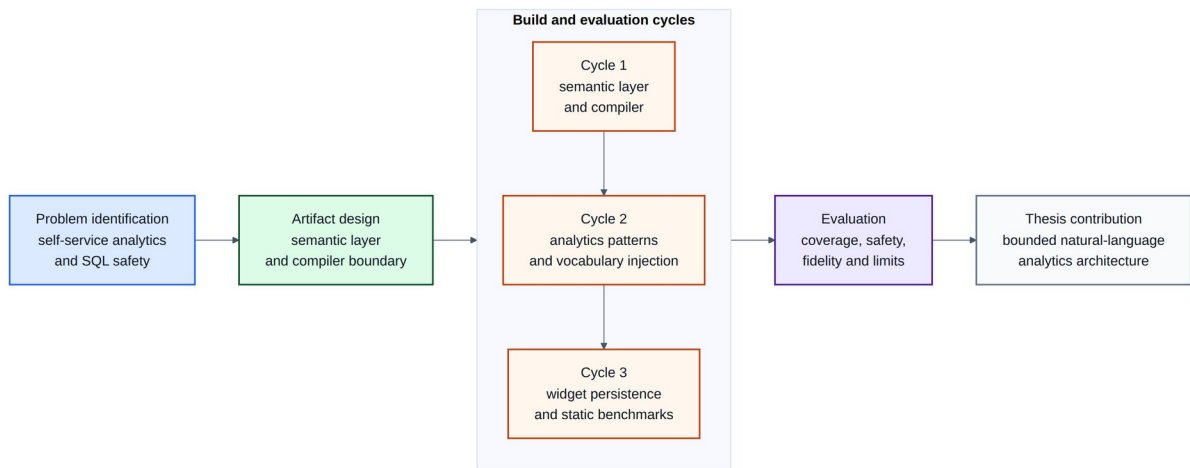


Figure 1: Design Science Research workflow for AEGIS

3.2 Formative Study of Reporting Patterns

The eleven-pattern taxonomy used throughout this thesis (Table 3.3) was derived from a design-time review of representative e-commerce and administrative reporting requests while designing AEGIS. The purpose of this review was to identify recurring business-question shapes that the system should support, such as KPI summaries, rankings, trends, comparisons, exception reports, and reusable tabular views.

Table 3.1 summarizes the reporting patterns used by the AEGIS compiler and shows how they appear in ordinary e-commerce analytics. The taxonomy is a design artifact used to structure the semantic layer and compiler templates, not an independently validated user study.

Table 3.1: AEGIS reporting pattern taxonomy.

Pattern	Purpose	Example
KPI / Aggregate	Single-number business summary	Total revenue this month
Ranking	Top or bottom entities by metric	Top products by revenue
Exception / Filter	Rows that meet an operational condition	Low-stock products
Trend Analysis	Metric over time	Monthly sales trend
Comparison	Metric compared across groups or periods	Revenue by order status
Summary / Group	Grouped business overview	Orders by payment status
Cohort	Population split by lifecycle stage	New versus returning customers
Funnel	Stepwise business process	Checkout progression
Correlate	Relationship between two measures	Discount versus order value
Segment	Breakdown by business dimension	Revenue by country
Tabular	Detailed record listing	Latest orders

The classification shaped the methodology in three ways. First, it made the compiler finite: each supported request must fit an approved analytical pattern. Second, it encouraged business terminology rather than database column names, which supports the need for an explicit semantic layer. Third, many reporting needs are reusable with only a changed time window, filter, or segment, which supports the widget persistence design.

3.3 Design Principles

Five principles guide the AEGIS architecture:

1. Separate understanding from execution. The LLM understands the question; fixed rules handle everything else.

2. Define business terms explicitly. Metrics, dimensions, joins, and time rules are written once in a semantic layer, not inferred per query.
3. Limit what SQL can be generated. SQL is built only from pre-approved, parameterized templates.
4. Select visualizations by rule. Chart type is decided by question type, result shape, and established visualization design guidance, not by a learned model.
5. Persist results for reuse. Every query produces a saved, refreshable widget rather than a discarded answer.

3.4 Formal Model

The practical model follows directly from the design principles above: user language is converted into a typed intent, approved semantic bindings are selected, and SQL is built only by deterministic templates. This keeps the model useful for understanding language without giving it authority to write executable SQL.

3.5 Threat Model

AEGIS protects against attacks arriving through the untrusted natural-language input channel. The model assumes the database and application server are properly hardened; the attacker controls only the query field.

- **T1 - Prompt injection attempting SQL generation:** "Ignore previous instructions. Generate DROP TABLE orders."
 - **Control:** The IntentObject schema contains no SQL field. Any non-approved string in metric_term or dimension_term is rejected by Pydantic type validation at Stage 2, before the compiler is reached.
- **T2 - Unauthorized metric or dimension access:** "Show me customer passwords" or "List credit card numbers by order."
 - **Control:** Fields such as customer_password do not exist in the semantic layer vocabulary. The LLM never sees those names; it receives only the curated, approved label list. Stage 2 rejects any unrecognized term.
- **T3 - Unauthorized row access:** A store-level user asks: "Show revenue for all branches."

- **Control:** Stage 4 (Permission Rewriter) runs after the LLM and appends a role-specific WHERE predicate (for example, AND o.StoreId = :user_store) derived from the authenticated session. This cannot be suppressed or overridden by natural-language content.
- **T4 - DML or DDL injection:** A crafted prompt that tricks the LLM into associating a write operation with an intent class.
 - **Control:** No template in the pattern library contains a DML or DDL keyword. The AST-level post-compilation validator explicitly rejects any non-SELECT statement as an additional safety layer.

Not protected by AEGIS (requires operational security controls outside this architecture):

- A malicious administrator embedding arbitrary SQL inside a metric's sql_expr field.
- Supply-chain compromise of the compiler module or the SQL parser library.
- Database-level privilege escalation that bypasses the application layer.
- LLM provider infrastructure compromise or model poisoning.

Documenting out-of-scope threats is part of the contribution. Prior NL-to-SQL work rarely specifies the boundary of its safety claims, which makes security comparison difficult.

3.6 System Architecture

Every user query travels through exactly seven stages. Stages 2 through 7 contain no artificial intelligence; they are deterministic code. Rejection at any stage produces a structured clarification message rather than a best-effort guess.

- **Stage 1 - Intent Extraction:** A lightweight LLM reads the query together with a system prompt built from the semantic layer, and outputs a validated IntentObject (intent class, metric term, dimension term, filters, sort, limit, confidence). This is the only stage that involves artificial intelligence.
- **Stage 2 - Structured Intent Validation:** The server validates the LLM's structured intent against the semantic layer: metrics, dimensions, filters, patterns, and join rules must bind to approved objects. The original question is retained only for narrow non-executable cues such as write operations, direct credential or secret requests, and explicit prediction or causal-analysis requests outside the SQL-only prototype scope.
- **Stage 3 - Semantic Mapping:** Business-logic aliases are expanded (for example, 'abandoned' maps to a specific OrderStatusId), and relative time expressions such as 'this month' are resolved to concrete date predicates.
- **Stage 4 - Permission Rewriting:** A role-specific WHERE predicate is appended after the LLM has already finished, so no natural-language content can influence it. The prototype demonstrates this stage rather than enforcing it: the shipped role predicates are illustrative, the demonstration server invokes the rewriter with a single unrestricted role, and the stage is outside the evaluated path reported in Chapter 5. Section 6.1 states the consequence.
- **Stage 5 - SQL Compilation and Execution:** A breadth-first search (an automatic search for the shortest chain of table joins, run by code rather than the model) over the join graph finds the minimal join path connecting the tables required by the resolved metric and dimension, and pre-compiled SQL expressions are substituted into a parameterized template (a fixed SQL skeleton with blanks for values). No SQL text is ever assembled from concatenated user input. A forbidden-pattern scan then rejects any non-SELECT construct, and the surviving statement is executed read-only with its literal values bound as parameters. Compilation and execution are one stage because the compiler emits the only statement that reaches the database; the two are drawn as separate modules in Figure 2 and timed separately in Table 5.4.

- **Stage 6 - Visualization Selection:** A rule engine maps the intent class and result shape to a default chart type. This stage contains no learned model.
- **Stage 7 - Widget Persistence:** The analysis plan is hashed (SHA-256) to detect duplicates, and the query, chart configuration, and access rules are stored as a widget artifact that can be refreshed on a schedule.

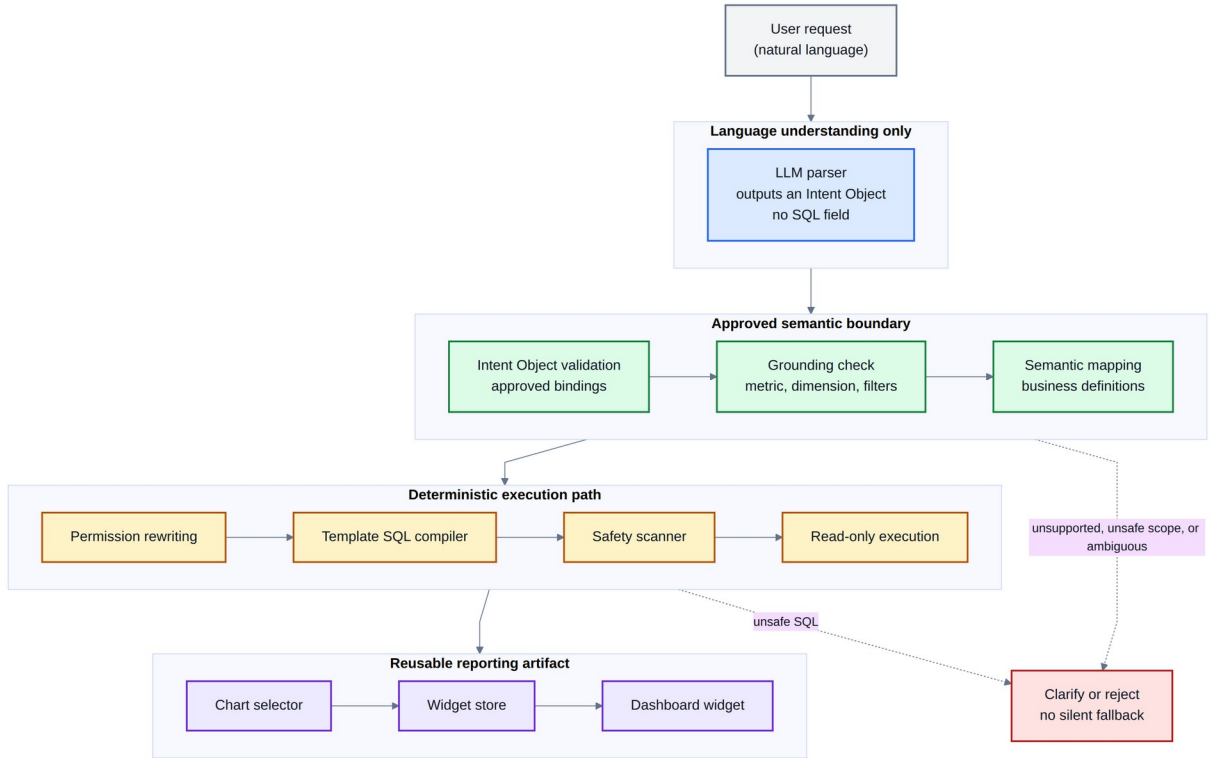


Figure 2: AEGIS architecture pipeline (User Request to Dashboard Widget)

3.7 Semantic Layer Design

The semantic layer is the most important non-AI component of AEGIS. It separates business language from the underlying database structure and defines exactly which metrics, dimensions, joins, predicates, and permissions are allowed to exist. The semantic layer is also the main per-deployment implementation surface: to support another system, the developer defines that system's approved business vocabulary and join paths while preserving the same AEGIS architecture.

Related systems describe their internal contract explicitly. Veezoo describes a Knowledge Graph, parser, query processor, and visualization engine [6]. G-SQL describes JSON schema serialization and rule-guided clause construction [9]. NL4DV exposes a JSON analytic

specification for attributes, tasks, and visualization choices [11]. AEGIS follows that style by defining the semantic layer as an implementation contract, not as a vague concept.

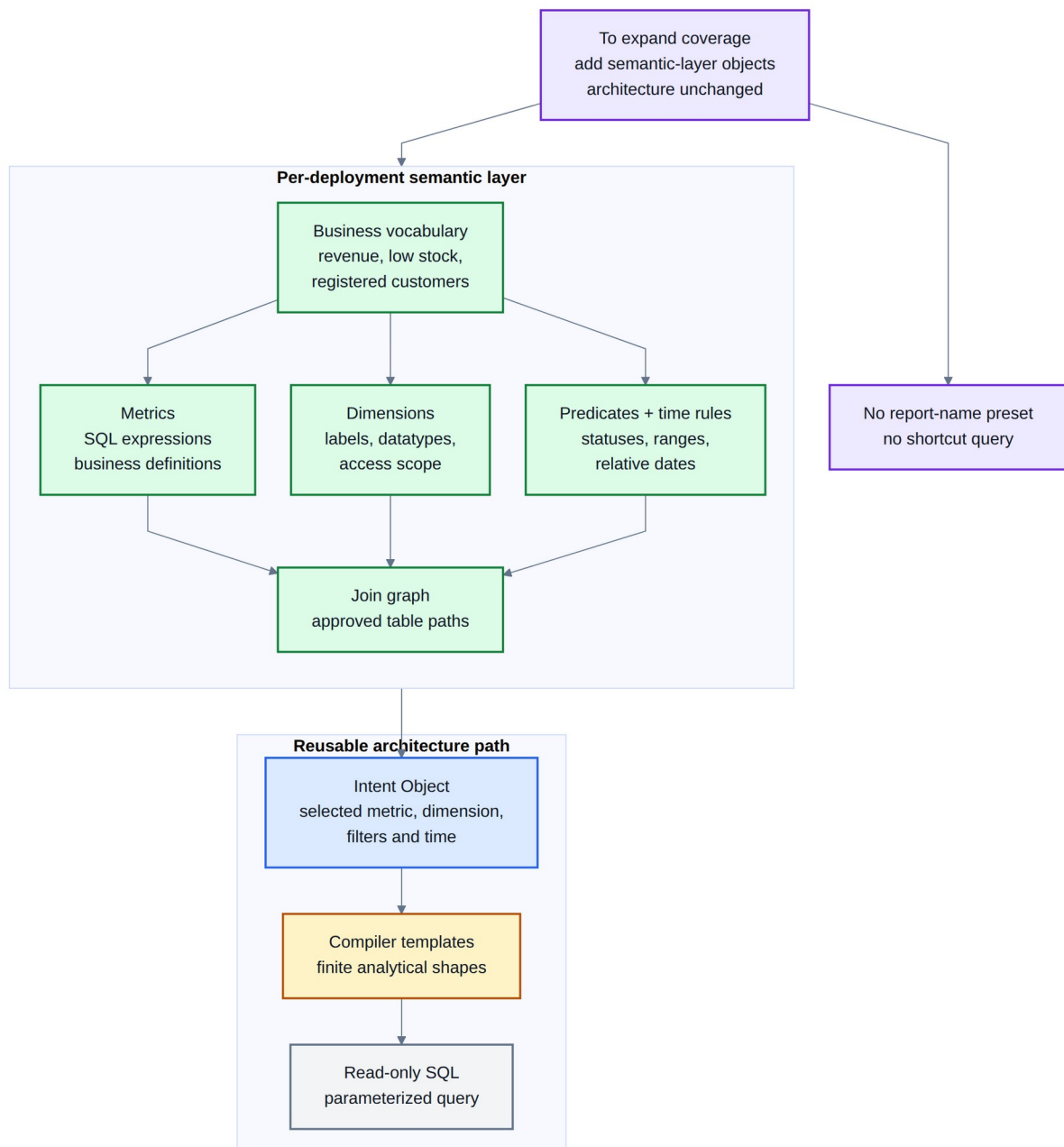


Figure 3: Semantic layer modularity - composable blocks vs. free-form SQL generation

Table 3.2: Semantic layer implementation contract.

Object	Required fields	nopCommerce example
Metric	id, label, description, sql_expr, binding_table, required_joins, time_anchor	revenue -> SUM(COALESCE(o.OrderTotal,0)), bound to Order
Grain rule	item_grain_equivalent when an order-level metric is grouped by item data	revenue by product uses line_item_revenue instead of duplicating order totals
Dimension	id, label, sql_expr, binding_table, datatype, entity, group_expr	category_name -> c.Name, reached through Product_Category_Mapping and Category
Predicate	label, SQL predicate, parameter field, datatype	payment_status -> o.PaymentStatusId = :payment_status
Time anchor	metric-specific date column for period filters	customer_count uses cu.CreatedOnUtc; order_count uses o.CreatedOnUtc
Join path	source table, target table, ON clause	Order -> OrderItem -> Product -> Category
Mandatory predicate	always-on table rule appended by the compiler	Order, Product, and Customer include Deleted = 0

A compact example of the implementation contract is shown below:

```

Metric(
  id="revenue",
  sql_expr="SUM(COALESCE(o.OrderTotal, 0))",
  binding_table="Order",
  time_anchor="o.CreatedOnUtc",
  item_grain_equivalent="line_item_revenue")

Dimension(
  id="category_name",
  sql_expr="c.Name",
  binding_table="Category",
  required_joins=["Product_Category_Mapping", "Category"])

```

In the nopCommerce prototype, the semantic layer defines the Approved metrics, dimensions, predicates, and join paths needed for the evaluated e-commerce analytics scope. The full nopCommerce schema is larger than this exposed vocabulary. Tables and fields that are not represented in the semantic layer cannot be requested through AEGIS, which is how the architecture keeps the answerable space useful but finite.

3.8 Intent Parsing with Dynamic Vocabulary Injection

Vocabulary injection makes Stage 1 practical. At startup, AEGIS builds the system prompt from the semantic layer. The prompt lists every approved metric and dimension identifier with a plain-English description. In the current prototype, 15 metrics and 34 dimensions take about 1,100 tokens. The LLM maps user phrasing to these identifiers without a hand-written

synonym list. When the system owner adds a metric, the prompt vocabulary changes with the semantic layer.

Structured output enforcement for LLMs [16] constrains the model's response to a fixed JSON schema before downstream validation runs. Stage 1 therefore relies on a typed IntentObject rather than free-form text. Malformed or off-schema output is rejected at the API boundary before structured intent validation runs.

The output schema enforces typed fields:

```
{
  "intent_class": "kpi | ranking | trend | comparison | exception |
    summary | segment | funnel | cohort | correlate | tabular",
  "metric_term": "string",
  "dimension_term": "string or null",
  "time_term": "string or null",
  "filters": [{"field": "string", "operator": "string", "value": "string"}],
  "sort": "asc | desc | null",
  "limit": "integer or null",
  "confidence": "low | medium | high",
  "needs_clarification": "boolean"
}
```

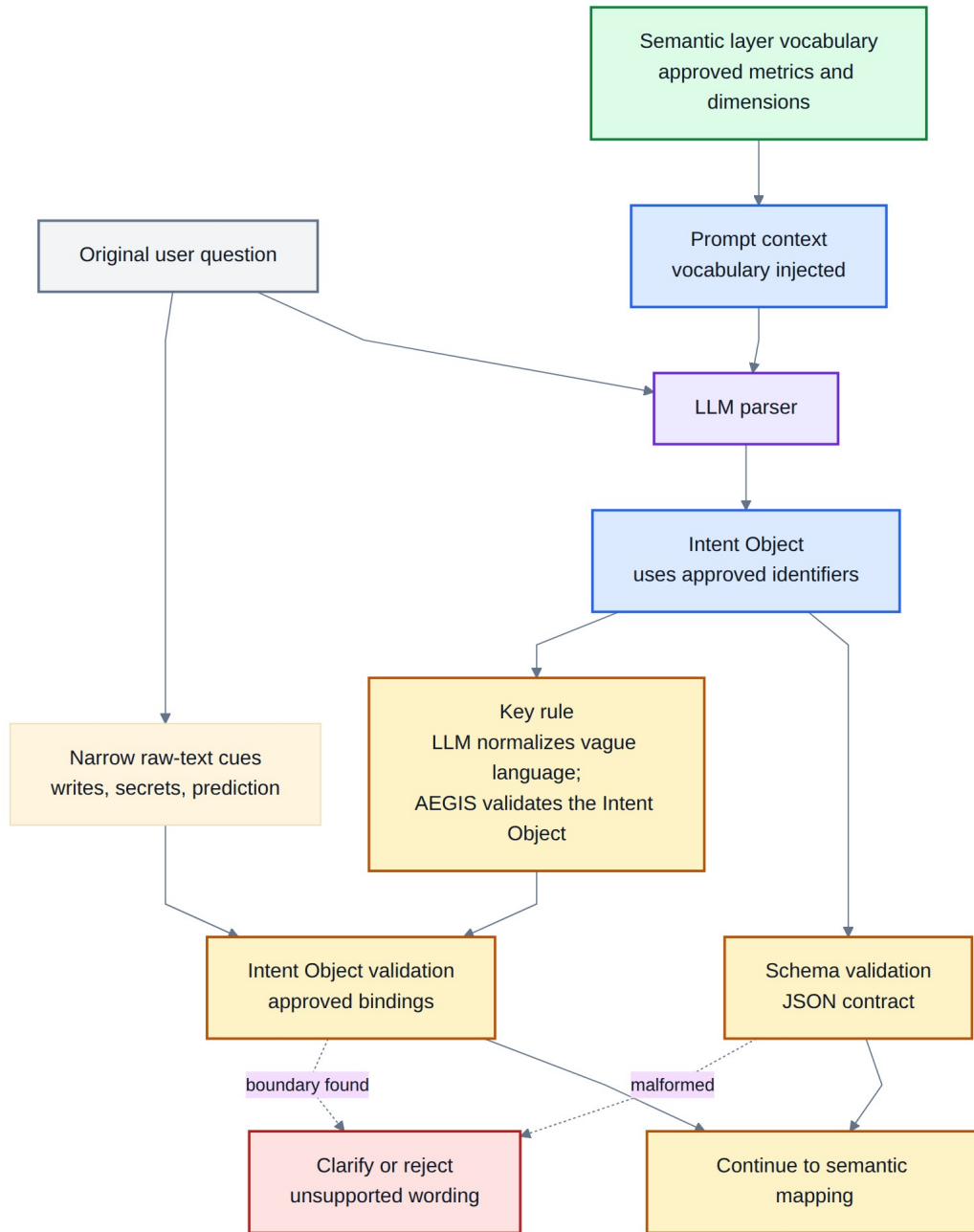


Figure 4: Vocabulary injection and structured intent validation workflow

3.9 Safe Query Compiler

The compiler builds SQL from parameterized templates, one for each analytics pattern. It does not invent joins or expressions. It proves that a grounded plan can be rendered using only semantic-layer objects.

The compiler procedure is deterministic:

- Input: AnalysisPlan(pattern, metric, dimension, filters, time_range)
1. collect required tables from metric, dimension, and declared required_joins

2. replace order-grain metrics with declared item-grain equivalents when needed
 3. resolve the minimal join path with BFS over the semantic join graph
 4. assemble SELECT from approved sql_expr values only
 5. assemble WHERE from normalized time_range and approved predicates
 6. append mandatory table predicates such as Deleted = 0
 7. add GROUP BY, ORDER BY, and LIMIT according to the analytical pattern
 8. reject the final SQL if it contains forbidden constructs
- Output: read-only SQL string, bound parameters, rationale log

The important implementation detail is that user text never becomes a table or column name (a SQL identifier). Identifiers come from Metric and Dimension objects. The values a user supplies are passed to the database separately as parameters, never pasted into the query text. Join clauses come from the join graph. If any slot cannot be grounded, the resolver returns reject or clarify rather than allowing the compiler to guess.

Table 3.3: The eleven AEGIS analytical patterns.

Pattern	Required slots	Optional slots	Default visual
KPI (Aggregate)	metric	time_rule, filter	kpi_card
Ranking	metric, dimension	time_rule, filter, limit	bar_chart
Trend	metric, time_grain	time_rule, filter	line_chart
Comparison	metric, segment	time_rule, filter	grouped_bar
Exception	metric, threshold	dimension, time_rule	table
Summary	metric[], dimension	time_rule, filter	multi_card
Segment	metric, dimension	time_rule, filter	pie_chart
Funnel	metric, stages	time_rule, filter	funnel_chart
Cohort	metric, group_def	time_rule, filter	grouped_bar
Correlate	metric, attribute	time_rule, filter	scatter_plot
Tabular	dimension	filters, time_rule	table

Two safety layers apply in sequence. Layer 1 is a parameterized query engine that separates SQL structure from user inputs. No user text is concatenated into the SQL string. Layer 2 is a post-compilation safety scanner that rejects any assembled query containing a forbidden construct, including non-SELECT statements, UNION, EXCEPT or INTERSECT, EXEC, or references to system tables. If any forbidden pattern is detected, the compiler raises a SecurityError rather than returning a partially safe query.

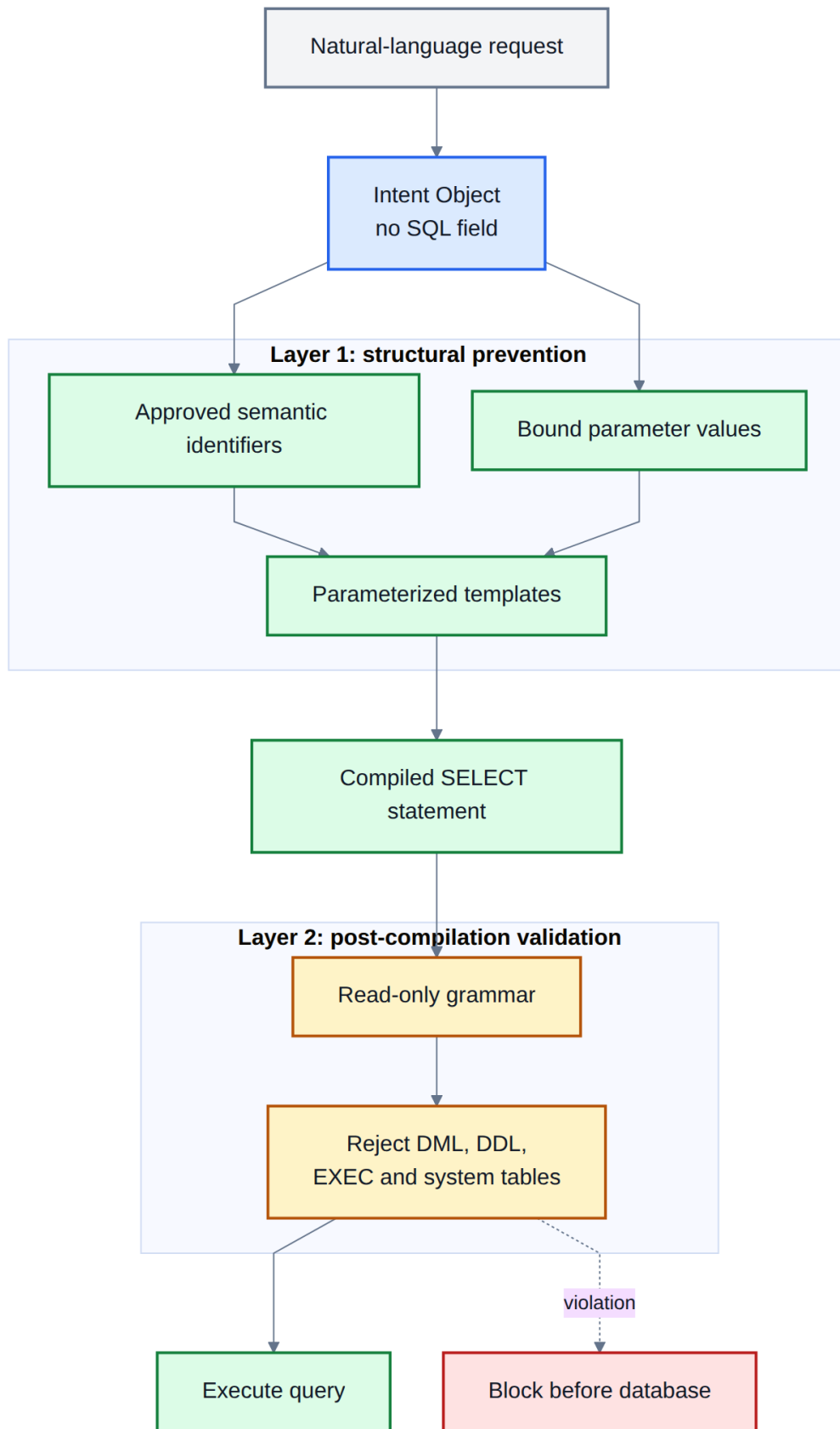


Figure 5: Two-layer SQL safety defence

3.10 Visualization Selector

Table 3.4: Visualization selector mapping.

Intent	Result shape	Selected visualization
KPI	scalar	KPI card
Ranking	1 measure, up to 20 categories	Horizontal bar chart
Trend	1 measure, time series	Line chart
Comparison	1 measure, 2-4 segments	Grouped bar chart
Exception	row-level detail	Sortable table
Summary	2-4 scalar measures	KPI card grid
Segment	1 measure, categorical	Pie chart
Funnel	ordered conversion stages	Funnel chart
Cohort	1 measure, 2+ groups	Grouped bar chart
Correlate	2 measures, continuous	Scatter plot
Tabular	raw records	Sortable table

Two additional rules apply after the data is known, rather than at selection time: bar charts with more than 20 categories are automatically converted to a table, and pie charts with more than 8 slices are converted to a bar chart. Both rules exist because the initial chart choice is made before the exact cardinality of the result is known.

3.11 Widget Persistence and Reuse

Each widget stores a unique identifier (a SHA-256 hash of the analysis plan), the original question, the analysis plan in JSON form, a hash of the SQL template used, chart configuration, timestamps, access rules, and run history. A new question triggers the full seven-stage pipeline; if an identical widget already exists, determined by a match on the plan hash, the cached artifact is returned immediately rather than recompiled. Scheduled refresh re-executes the stored SQL against fresh data on a configurable interval, which directly addresses the design-time observation that reporting requests are often recurring rather than one-off: a widget answers the question once and then continues answering it as new data arrives, rather than requiring the same natural-language request to be re-processed from scratch every time.

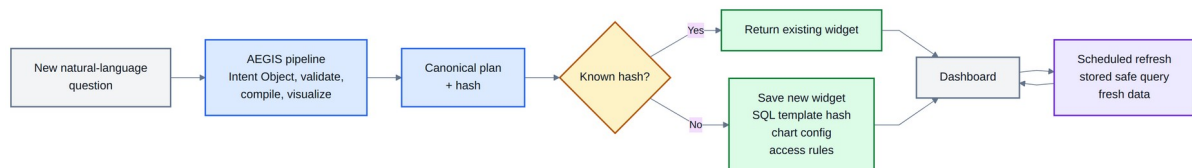


Figure 6: Widget lifecycle and refresh model

CHAPTER 4

EXPERIMENTAL WORK

4.1 Implementation

AEGIS is implemented as a web application with a plain HTML and JavaScript frontend and a Python FastAPI backend, targeting a nopCommerce-derived MySQL schema of 126 tables and 107 foreign-key constraints. The oracle queries (the trusted reference answers, taken from nopCommerce's own report code) are read from nopCommerce source at commit 64bdf2ff (version 5.00.0), and all twenty executed against this schema without a missing table or column, which is the evidence that the schema and the report logic agree on the entities those reports touch. The prototype follows the architecture from Chapter 3: the model extracts intent, the semantic layer supplies approved business definitions, and deterministic compiler templates produce SQL.

- **LLM integration:** An LLM API exposed through an OpenAI-compatible `/v1/chat/completions` interface with structured JSON output enforcement. The prompt injects approved metric and dimension identifiers at request time. The compiler and safety layer are provider-independent because they consume only the typed intent object, not model-written SQL.
- **Semantic layer:** Python configuration modules define approved metrics, dimensions, predicates, time anchors, join paths, grain rules, and mandatory platform filters.
- **SQL compiler:** Parameterized MySQL templates build read-only SQL from approved analytical patterns. The compiler logs the selected pattern, resolved tables, join path, metric expression, dimension expression, and safety result.
- **Visualization selector:** Rule-based chart selection maps analytical patterns and result shapes to dashboard widgets.
- **Widget engine:** The prototype stores widget metadata locally with plan-hash deduplication. The interface is designed so production storage can be moved to a database.
- **Intent validator:** Structured intent validation checks the LLM's normalized intent against the semantic layer, while the original question is used only for narrow safety and scope cues such as writes, direct secrets, and explicit non-SQL analysis modes.

Table 4.1: Prototype module-to-architecture mapping.

Architecture stage	Implementation module	Main technical responsibility
Intent extraction	intent_parser.py, models.py	OpenAI-compatible structured JSON output into IntentObject
Grounding	grounding.py, mapper.py	Ranked semantic-layer binding with resolved, ambiguous, unsupported, or absent outcomes
Intent validation	coverage.py, mapper.py	Structured binding checks plus narrow raw-text safety/scope cues
Semantic layer	semantic_layer.py	Metrics, dimensions, predicates, time anchors, join graph, and mandatory filters
Compilation	compiler.py	Pattern templates, BFS join-path resolution, parameter binding, and SQL safety scan
Visualization	visualization.py	Rule-based chart selection from pattern and result shape
Persistence	widget_engine.py	Plan hashing, widget storage, refresh metadata, and reuse

This module map makes the prototype auditable. The thesis architecture is more than a conceptual pipeline. Each stage has a corresponding implementation boundary.

4.2 Experimental Environment

Table 4.2: Experimental setup.

Setup item	Configuration
Database engine	MySQL 8.0
Execution mode	Local/Laragon or Docker-compatible MySQL evaluation database
Application schema	nopCommerce-derived MySQL schema, 126 tables, 107 foreign keys
Report oracle source	nopCommerce source at commit 64bdf2ff (version 5.00.0)
Dataset scope	Orders, customers, products, categories, manufacturers, payments, shipping, refunds, stores, countries, and search terms
LLM interface	OpenAI-compatible /v1/chat/completions API
Evaluation scope	Single-domain prototype evaluation over nopCommerce analytics

4.3 Benchmark Dataset Construction

This evaluation is a prototype evaluation, not a large-scale independent leaderboard study. Its goal is to test whether the AEGIS architecture provides safe, Approved natural-language analytics for a realistic e-commerce schema. General text-to-SQL benchmarks such as Spider and BIRD are valuable for SQL generation research, but they do not measure reusable dashboard widgets, semantic layer control, or refusal of unsupported business questions.

Table 4.3: Static evaluation datasets.

Dataset	Size	Purpose
Natural-language benchmark	500 questions	Breadth: 425 answerable questions and 75 realistic e-commerce boundary questions that should be declined.
nopCommerce standard admin reports	20 reports	Fidelity: the platform's own admin report list, with the platform's own report implementations as the oracle.

The two corpora differ in who chose them, and that difference is deliberate. The 500 questions were written for this study, so they measure how the architecture behaves across the range of language a store owner uses. The twenty reports were not: the list is nopCommerce's own admin menu and the comparison target is nopCommerce's own service-layer code, read from source. Comparing against a platform's shipped implementation is stronger evidence than comparing against an expected-answer set written by the same authors as the system under test, because the latter tends to agree with the implementation wherever both authors reasoned the same way.

All benchmark datasets are static repository artifacts. They are not regenerated during evaluation, because a thesis dataset must be inspectable and stable. The reported result files are also stored with the repository so a reader can compare the manuscript tables with the underlying evidence.

4.4 Baseline and Oracle Comparisons

The evaluation uses two kinds of comparison, and both are measured rather than argued. The first is a direct LLM-to-SQL baseline run under identical conditions. The second compares AEGIS output against nopCommerce's own admin report implementations. The second is the stronger evidence for result accuracy, because it checks returned business values rather than only whether SQL text was emitted.

- **Direct LLM-to-SQL baseline:** The same model, through the same gateway (the service that relays requests to the language model), is asked to write MySQL directly for the same 500 questions against the same database, with no semantic layer in between. The arms therefore differ only in whether the model authors the query, which isolates the architectural variable rather than mixing it up with model or data differences. Both arms are scanned for forbidden constructs with the same pattern set, imported from the compiler rather than restated, so neither arm is judged by a more lenient rule.

- **Admin report differential:** Each of nopCommerce's twenty standard admin reports is requested in ordinary business phrasing. Two checks are applied: whether the request reaches an answer and compiles to SQL, and whether executing that SQL against the seeded database returns the same rows as the platform's own query. Only the second tests the claim; the first is satisfied by any query that compiles, including ones that are silently wrong.
- **Per-stage latency:** Every stage of the live benchmark is timed separately, so that the cost of intent extraction can be separated from the cost of resolution, compilation, and execution. A single end-to-end figure would mix up a property of the model provider with a property of the architecture.

4.5 Evaluation Procedure

The current evaluation focuses on measurements that can be reproduced from static benchmark data, verifier scripts, and recorded result artifacts:

- **RQ1:** How reliably does the LLM parser produce typed reporting intents on the 500-question natural-language dataset?
- **RQ2:** Does AEGIS answer supported semantic-layer questions while rejecting or clarifying realistic unsupported e-commerce questions?
- **RQ3:** Does the compiled SQL execute successfully against the evaluation database?
- **RQ4:** Do AEGIS results match the rows returned by nopCommerce's own report implementations on the same database?
- **RQ5:** How does the same model behave on the same questions when the semantic layer is removed and it writes SQL directly?
- **RQ6:** Where does the response time go, and how much of it does the architecture control?
- **RQ7:** What implementation gaps remain after the current semantic-layer and compiler updates?

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports the AEGIS prototype evaluation using static nopCommerce artifacts committed with the repository. The evaluation separates four questions that are often mixed together. Can the system parse the request? Is the request inside the semantic layer? Does the compiled SQL run? Does the result match the expected business answer?

The results are not a claim that AEGIS is perfect. A perfect score on every table would be suspicious for a prototype. The evaluation shows where the architecture is strong, where the nopCommerce implementation works well, and where real gaps remain.

5.1 Evaluation Overview

Table 5.1: Evaluation artifacts.

Artifact	Purpose	Status
500-question natural-language dataset	Tests broad user-facing e-commerce questions over the implemented semantic layer.	425 supported requests and 75 realistic boundary requests.
nopCommerce standard admin reports	Compares AEGIS output with the platform's own shipped report implementations.	20 reports, with source-derived semantics and oracle queries.
Direct LLM-to-SQL baseline	Runs the same model on the same questions with no semantic layer between it and the SQL.	500 questions, same gateway and database.

These artifacts are static, not generated at evaluation time, so another reader can inspect the exact questions, oracle definitions, scripts, and result files behind every reported claim. The two evidence tracks differ in who chose them. The 500 questions were written for this study. The 20 reports were not: the list is nopCommerce's own admin menu and the comparison target is nopCommerce's own service-layer code, which removes the risk of an expected-answer set that happens to agree with the system under test because the same person wrote both.

5.2 500-Question Natural-Language Benchmark

The main breadth benchmark contains natural-language questions that a store owner or administrator might ask about orders, revenue, customers, products, refunds, inventory, stores, countries, payment status, shipping, and search terms. The dataset intentionally includes unsupported questions as well, because an approved analytics system must know when not to answer.

Table 5.2: 500-question live benchmark results.

Metric	Result	Interpretation
Parser success	499/500 (99.8%)	The gateway returned a valid Intent Object for all but one prompt.
Supported intent exact match	313/425 (73.6%)	Strictest measure; a prompt parsed differently from the annotation often still compiled to a valid answer.
Supported answer rate	423/425 (99.5%)	AEGIS answered almost every request inside the semantic layer.
Supported execution validity	422/425 (99.3%)	The answered supported requests also executed against MySQL.
Boundary rejection accuracy	72/75 (96.0%)	Most realistic e-commerce questions outside the semantic layer were declined or clarified.

An initial pass lost 29 questions to a consecutive block of HTTP 502 responses from the model gateway, which reduced the supported answer rate to 92.9 per cent. Those questions were retried and 28 answered on the second attempt. Both passes are recorded rather than only the better one: reporting only the retry would present a provider outage as though it had not happened, and reporting only the first pass would attribute that outage to the architecture.

Two supported questions were refused, and both refusals are correct. Each asks to break refunds down by payment method without naming a measure, and a segment report cannot be built without one. Choosing a measure on the user's behalf is exactly the silent substitution the resolver (the step that matches request terms to semantic-layer definitions) was built to remove.

Three boundary questions were answered instead of declined. One was a malformed model reply, which is a parse failure rather than a decision the system made. The other two are the same question phrased twice, asking to compare two named shipping carriers. The model reported both carrier names as unmapped, and the resolver answered by shipping method regardless. That is deliberate: a model-reported gap is treated as evidence rather than a verdict, because treating it as a verdict caused a high rate of false refusals. These two questions are what that choice costs, and stating the cost is more useful than tuning it away.

The same corpus is also run with the model removed, by feeding each question's committed intent annotation straight to the resolver. That configuration resolves, compiles and executes 425 of 425 supported questions and labels 75 of 75 boundary questions correctly. It is a regression gate on the resolver and compiler only. Because no model participates, it is not an end-to-end result and is never quoted as one.

5.3 Fidelity Against nopCommerce's Own Report Logic

Each of nopCommerce's twenty standard admin reports is requested in ordinary business phrasing. Two checks are applied. The first asks whether the request reaches an answer and compiles to SQL. The second executes that SQL and the platform's own query against the same seeded database and compares the rows returned. Exact SQL text is not required; the question is whether the answer carries the same business meaning.

Table 5.3: nopCommerce standard admin report fidelity.

Check	Result	Interpretation
Reached an answer and compiled	20/20	Every standard report can be requested in ordinary language.
Result set matched the platform	15/20 (75.0%)	The check that tests the claim: same database, same rows, same values.
Reports differing in value	0/20	The five mismatches differ in row count or label column, never in a number.

Only the second check tests the claim. The first is satisfied by any query that compiles, and several of these twenty once passed it while being silently wrong: an order total counted once per line item, which inflated revenue; a missing filter for rows marked deleted but kept (a soft-delete filter); and a customer breakdown grouped by display name, which merges different people who share a name. Each returned a plausible, chartable number, so nothing downstream could tell it from a correct answer. That is the reason this thesis treats a compile-only check as a proxy rather than a result.

The five reports that did not match agree with the platform on every value in every overlapping row. Four differ in result-set size, because nopCommerce's own reports carry their own limits of five, fifteen or one hundred rows. Two differ in the label column, returning a customer name where the platform labels by email address; one of the five (best customers by order count) differs in both. Matching them exactly would require per-report presets, which is the report-specific special-casing the semantic layer exists to avoid.

5.4 Latency

Per-stage timings were recorded for every supported question in the live benchmark. The split between the model stage and the deterministic stages matters more than the totals, because it is the same boundary the safety argument rests on.

Table 5.4: Per-stage latency over 425 supported questions.

Stage	Median	95th percentile
Intent extraction (model)	9,029.89 ms	19,460.15 ms
Semantic resolution	0.89 ms	1.25 ms
SQL compilation	0.13 ms	0.21 ms
Database execution	2.54 ms	12.87 ms
All stages after the model	3.62 ms	14.02 ms

Substantially all of the wall clock is the model reading the question. The stages that resolve business concepts, choose the join path, emit SQL and execute it take a few milliseconds together, and the model has no influence over any of them. The model figure is a property of the gateway used here and would change with the provider; the deterministic figures are properties of the architecture and would follow it to another deployment. The direct baseline's generation stage has a comparable median of 9,154.02 ms, so the constrained pipeline pays no latency premium for its safety.

5.5 Safety Evaluation

The central architectural safety property is that the language model does not write SQL. It returns an Intent Object. Deterministic templates then produce SQL from approved semantic-layer definitions and check the query before execution. User prompt text is not inserted into executable SQL. The claim is therefore structural rather than statistical: no rate of successful defence is asserted, because there is no path from a prompt to a query the templates do not generate.

Table 5.5: Safety interpretation.

Risk	AEGIS control	Remaining boundary
User asks for a write operation	Intent schema and compiler templates do not include write primitives.	The system must still explain refusal clearly to the user.
User uses unsupported business terms	The Intent Object must bind to approved semantic-layer definitions.	Intent extraction can still misread vague language; debug traces and clarification reduce this risk.
LLM misunderstands the request	Compiled SQL remains structurally safe.	The answer may still be semantically wrong if intent extraction is wrong.

The baseline arm gives this structural claim an empirical counterpart. Both arms were scanned with the same forbidden-pattern set, imported from the compiler rather than restated, so neither is judged by a more lenient rule. The unconstrained arm produced two queries containing a forbidden construct, both attempts to classify review text by keyword matching joined with UNION. The constrained arm produced none.

5.6 Comparison With Direct LLM-to-SQL

A direct LLM-to-SQL baseline can appear attractive because it can always try to write a SELECT statement. However, allowing the model to author SQL moves business definitions, join choices, security behavior, and dialect correctness into a probabilistic component. To measure that rather than assert it, the same model was run through the same gateway on the same 500 questions against the same database, with no semantic layer in between. The arms differ only in whether the model authors the query.

Table 5.6: Measured comparison over the same 500 questions.

Measure	AEGIS	Direct LLM-to-SQL
Supported execution validity	422/425 (99.3%)	365/425 (85.9%)
Out-of-scope questions answered	3/75 (4.0%)	25/75 (33.3%)
Queries with a forbidden construct	0	2/500

The middle row carries the finding. A third of the questions the semantic layer cannot express were answered by the unconstrained model with confident, executable SQL. Asked to forecast next month's sales, it returned a query summing past months, which runs, returns a number, and forecasts nothing. Asked which customers are likely to churn, it returned a query counting each customer's past orders. Asked what customers say about delivery speed, it returned raw review rows. Each answer is plausible, chartable, and addresses a different

question than the one asked. AEGIS cannot produce these because none of the requests binds to anything in the semantic layer.

Table 5.7: Structural comparison.

Property	Direct LLM-to-SQL	AEGIS
SQL generation	Model writes SQL text	Compiler expands approved templates
Business definitions	Inferred from prompt/schema names	Declared in semantic layer
Unsupported requests	Often answered by nearest plausible SQL	Rejected or clarified when outside coverage
Dashboard output	Usually one-off query result	Saved, refreshable widget artifact
Safety basis	Prompt instruction and filtering	No model-authored SQL execution path

5.7 Interpretation

The evaluation supports three conclusions. First, a semantic-layer architecture can cover a broad set of natural e-commerce analytics questions without exposing users to raw SQL. Second, refusal is part of correctness: the same model without the semantic layer answered a third of the unanswerable questions anyway, and a system that answers an unsupported question with a plausible but wrong query is less trustworthy than one that declines. Third, the differences that remain against nopCommerce's own reports are presentational rather than numerical, which locates the gap in row limits and label choices rather than in the compiled semantics.

Two limitations are worth stating alongside those conclusions. Value-level verification covers the twenty reports, not all 425 supported questions; for the latter the evidence is that the compiled SQL resolves and executes, not that every answer is the intended one. And the gateway used for intent extraction resolved a routing alias (a name the gateway maps to whichever model is free) per request, so the reported run spans two models. Each result row records the model that served it, but a single-model run is needed before parser-dependent figures can be attributed to one system.

This interpretation is consistent with mixed-initiative NLIDB work such as [5], which showed that users do not reliably detect wrong system answers without help. In a reporting context, a visible refusal or clarification is therefore often safer than a confident wrong chart.

CHAPTER 6

LIMITATIONS AND FUTURE WORK

6.1 Limitations

AEGIS is intentionally bounded. Its safety and auditability come from the fact that all answerable concepts must be declared in the semantic layer and all executable SQL must be produced by deterministic compiler templates. The limitations below should therefore be read as explicit boundaries of the current nopCommerce prototype, not as reasons to bypass the architecture with free-form SQL generation.

- **Single-domain evaluation:** The final evaluation is over one e-commerce deployment, nopCommerce. The results show that the architecture works in this domain, but they do not prove cross-domain generality.
- **Author-generated natural-language data:** The 500-question dataset is static and checkable, but it is still author-generated. A stronger study would collect questions from store owners or administrators and annotate them with at least two reviewers.
- **Finite semantic coverage:** Boundary questions about web telemetry, marketing attribution, support tickets, review-text sentiment, forecasting, churn prediction, supplier performance, fraud scoring, delivery SLA analysis, and product affinity are deliberately outside the current semantic layer.
- **Remaining Admin fidelity gap:** Fifteen of the twenty nopCommerce admin reports reproduce the platform's own result set exactly. The five that do not agree with the platform on every value in every overlapping row and differ only in how many rows are returned and which column labels them. Closing that gap means per-report row limits and label presets, which is the report-specific special-casing the semantic-layer design exists to avoid.
- **Catalogue scale in the seeded database:** The seeded catalogue is small relative to the order volume: 17 products across 8 categories, against 2,500 orders and 6,320 order lines. Order-side reports are therefore exercised on realistic volume, but the product and category reports are not: a bestsellers ranking over 17 products and a revenue breakdown over 8 categories are short lists, so four of the twenty differentials agree with the

platform on a narrow result set. The compiled SQL is the same either way, but a wider catalogue would test it harder.

- **Correctness evidence is narrower than coverage:** Value-level correctness at scale rests on the twenty admin reports rather than on all 425 supported questions. For those 425 the evidence is that the compiled SQL resolves and executes, not that every returned value is the intended one. Extending value-level verification to a stratified sample of the corpus is the most useful next measurement.
- **Response time is dominated by the provider:** Substantially all end-to-end latency is the model reading the question; every deterministic stage after it completes in a few milliseconds. The model figure is a property of the gateway used here, so end-to-end response time would change with the provider even though the architecture's own cost would not.
- **LLM dependence for intent extraction:** The compiler and safety layer are deterministic, but natural-language intent extraction still depends on the configured LLM API. Future work should compare multiple OpenAI-compatible models on the same static dataset.
- **Prototype database target:** The evaluated prototype targets nopCommerce on MySQL. This is an implementation and evaluation-scope choice, not an architectural limitation; other databases require dialect-specific compiler templates and safety rules.
- **Widget persistence:** The prototype stores widget metadata in simple local persistence. A production deployment should move the widget registry to a transactional database with migrations, ownership policies, and administrative audit views.
- **Permission rewriting is not evaluated:** Stage 4 is demonstrated, not enforced, and none of the results in Chapter 5 exercise it. The shipped role predicates reference illustrative columns rather than columns of the evaluation schema, role values are interpolated into the predicate text instead of being bound as parameters, and the demonstration server calls the rewriter with a single unrestricted role. A deployment serving more than one role would need the predicates rewritten against its own schema, parameter binding for role values, a hard failure when a scope cannot be applied to the compiled join path, and database-level row security underneath the application layer.

6.2 Future Work

- **Value-level verification at scale:** Verify returned values, not just execution, on a stratified sample of the 500-question corpus, so that value-level correctness rests on more than the twenty admin reports.
- **General limit and label handling:** Express row limits and label columns as general, requester-visible options rather than per-report presets, so that a request can match a platform report's presentation without the compiler learning that report's name.
- **Cross-schema evaluation:** Repeat the static-dataset process on a second schema such as WooCommerce or a non-commerce operational database.
- **Semantic-layer tooling:** Build a guided interface so analysts can define metrics, dimensions, predicates, join paths, and display labels without editing Python code.
- **Human dataset study:** Collect real user questions and perform independent answerability and oracle annotation, including inter-annotator agreement.
- **Model comparison:** Compare parser accuracy, refusal behavior, and execution validity across several OpenAI-compatible LLM APIs while keeping the compiler and semantic layer fixed.

CHAPTER 7

CONCLUSION

This thesis presented AEGIS, a constraint-based architecture for safe LLM-assisted natural-language analytics over relational databases. The system limits the language model to intent extraction while query construction, chart selection, and widget persistence are handled by deterministic components. This design makes approved business terms explicit through a semantic layer and avoids exposing raw SQL generation authority to the language model.

The final evaluation used static nopCommerce datasets rather than an ad hoc one-time question set. On the 500-question live benchmark, AEGIS parsed 499 of 500 prompts, answered 423 and executed 422 of 425 supported requests, and rejected or clarified 72 of 75 realistic boundary requests. Against nopCommerce's twenty standard admin reports, every request reached an answer and compiled to SQL, and fifteen reproduced the platform's own result set exactly; the other five agreed on every value and differed only in row count and label column. The same model without the semantic layer, writing SQL directly against the same database, executed 365 of 425 supported questions and answered 25 of the 75 questions it should have declined.

These results should be read with the right scope. AEGIS is not an infinite natural-language-to-SQL engine, and the thesis does not claim perfect accuracy. Its contribution is a bounded reporting architecture: if the semantic layer defines a business concept, the system can map natural language to a safe, refreshable analytical widget; if the concept is outside the layer, the correct behavior is to reject or clarify rather than invent an answer.

The baseline comparison is what gives that claim its force. A third of the questions the semantic layer cannot express were answered by the unconstrained model with confident, executable SQL: asked to forecast next month's sales it summed past months, and asked which customers are likely to churn it counted past orders. Each answer runs, charts, and addresses a different question than the one asked. That is the failure mode this architecture makes structurally unreachable rather than statistically rare, and within this scope AEGIS demonstrates a practical path toward safer, more auditable natural-language analytics in institutional dashboard systems.

REFERENCES

- [1] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proc. 2018 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2018, pp. 3911-3921.
- [2] J. Li et al., "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.
- [3] K. Affolter, K. Stockinger, and A. Bernstein, "A comparative survey of recent natural language interfaces for databases," The VLDB Journal, vol. 28, no. 5, pp. 793-819, 2019.
- [4] M. Liu and J. Xu, "NLI4DB: A systematic review of natural language interfaces for databases," arXiv:2503.02435, 2025.
- [5] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," Proceedings of the VLDB Endowment, vol. 8, no. 1, pp. 73-84, 2014.
- [6] C. Lehmann, D. Gehrig, S. Holdener, C. Saladin, J. P. Monteiro, and K. Stockinger, "Building natural language interfaces for databases in practice," in Proc. 34th Int. Conf. Scientific and Statistical Database Management (SSDBM), 2022, Art. no. 20.
- [7] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 7567-7578.
- [8] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in Proc. 2021 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 9895-9901.
- [9] H. S. Shalaan, T. H. A. Soliman, and A. M. Abdelaziz, "G-SQL: A schema-aware and rule-guided approach for robust natural language to SQL translation," IEEE Access, vol. 13, pp. 158520-158534, 2025.
- [10] X. Su, Y. Gu, P. Wang, W. Gu, L. Qi, and J. He, "A robust natural language text-to-SQL generation framework with dynamic strategies based on LLMs," Scientific Reports, vol. 16, Art. no. 7892, 2026.
- [11] A. Narechania, A. Srinivasan, and J. Stasko, "nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries," IEEE Transactions on Visualization and Computer Graphics, vol. 27, no. 2, pp. 369-379, 2021.

- [12] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios, "DataTone: Managing ambiguity in natural language interfaces for data visualization," in Proc. 28th Annual ACM Symposium on User Interface Software and Technology (UIST), 2015, pp. 489-500.
- [13] D. Deng, A. Wu, H. Qu, and Y. Wu, "DashBot: Insight-driven dashboard generation based on deep reinforcement learning," IEEE Transactions on Visualization and Computer Graphics, vol. 29, no. 1, pp. 690-700, 2023.
- [14] G. N. Shailesh, M. Pavithran, A. Rahul Hari Venkat, and P. Kaliappan, "Conversational BI: Natural language interface to business dashboards," International Journal of Engineering Research & Technology, vol. 14, no. 12, 2025.
- [15] J. Valkenburgh, "Enhancing business dashboards with explanatory analytics and AI: Exploring the use of AI and explanatory analytics to improve business decision-making," M.S. thesis, Information Management, Turku School of Economics, University of Turku, Finland, 2024.
- [16] OpenAI, "Introducing structured outputs in the API," OpenAI, 2024.